

ΠΑΡΑΛΛΗΛΑ ΣΥΣΤΗΜΑΤΑ

ΑΣΚΗΣΗ-2 Α



AM: 161173

Ονοματεπώνυμο: Βεσέλ Νταλίπαϊ

Το πρόγραμμα ξεκινάει διαβάζοντας από τις παραμέτρους εκτέλεσης. Πρώτα το N (μέγεθος του πίνακα A) και μετά τα `threads`. Αν λείπουν παράμετροι ή αν το $N/\text{threads}$ είναι μη έγκυρα (π.χ. $N \leq 0$ ή $\text{threads} \leq 0$), τερματίζει.

Στη συνέχεια δεσμεύεται μνήμη για δύο πίνακες: τον A που θα ταξινομηθεί και τον `tmp` που χρησιμοποιείται σαν βοηθητικός χώρος για τα merges. Αν αποτύχει κάποιο `malloc`, το πρόγραμμα τερματίζει.

Μετά αρχικοποιείται το random generator και καλείται η `generate_list`. Η `generate_list` γεμίζει τον πίνακα με τιμές $0..N-1$ και μετά τις ανακατεύει τυχαία.

Το πρόγραμμα τυπώνει τον πίνακα A πριν την ταξινόμηση, για να φαίνεται ότι όντως δεν είναι ταξινομημένος.

Έπειτα ορίζεται ο αριθμός νημάτων με `omp_set_num_threads` και ξεκινάει η χρονομέτρηση με `omp_get_wtime`.

Η ταξινόμηση γίνεται μέσα σε `#pragma omp parallel` ώστε να δημιουργηθεί ομάδα νημάτων, αλλά η εκκίνηση της αναδρομής γίνεται με `#pragma omp single` ώστε μόνο ένα νήμα να καλέσει την `multisort` και από εκεί και πέρα να φτιάξει `tasks`.

Η `multisort` υλοποιεί ακριβώς τη λογική του `multisort`. Αν το τμήμα που πρέπει να ταξινομηθεί είναι μικρό δεν συνεχίζει άλλο σε διαχωρισμό και πέφτει σε απλή `mergesort`. Το `limit` είναι ένα όριο που ελέγχει πότε σταματάει ο αναδρομικός διαχωρισμός.

Αν το size είναι μεγάλο, η multisort χωρίζει το τμήμα σε 4 υποτμήματα (A, B, C, D) με μέγεθος περίπου $\text{size}/4$ το καθένα. Για το τελευταίο κομμάτι (D) κρατάει το “υπόλοιπο” ώστε να καλυφθούν όλα τα στοιχεία.

Μετά δημιουργεί 4 OpenMP tasks ώστε τα 4 κομμάτια να ταξινομηθούν παράλληλα με αναδρομικές κλήσεις multisort. Ακολουθεί `#pragma omp taskwait` για να βεβαιωθεί ότι και τα 4 υποτμήματα έχουν επιστρέψει ταξινομημένα πριν ξεκινήσει η συγχώνευση.

Η συγχώνευση γίνεται σε δύο στάδια, πρώτα συγχωνεύει παράλληλα τα A,B σε έναν ενδιάμεσο χώρο και τα C,D σε έναν άλλο ενδιάμεσο χώρο χρησιμοποιώντας δύο tasks που καλούν τη `merge_halves`. Μετά από ένα δεύτερο `taskwait`, γίνεται η τελική συγχώνευση των δύο ενδιάμεσων ταξινομημένων κομματιών μέσα στον αρχικό πίνακα, ώστε το συνολικό τμήμα να καταλήξει ταξινομημένο.

Η `merge_halves` είναι μια `merge` με pointers, δηλαδή παίρνει δύο ήδη ταξινομημένα ranges και τα γράφει συγχωνευμένα στο `dest`.

Η `mergesort` χωρίζει το array σε δύο μισά, δημιουργεί 2 tasks για να ταξινομηθούν τα μισά, περιμένει με `taskwait` και μετά τα ενώνει με τη `merge`. Η `merge` ενώνει τα δύο μισά του ίδιου πίνακα χρησιμοποιώντας τον βοηθητικό πίνακα `tmp` και στο τέλος κάνει `memcpy` πίσω.

Όταν ολοκληρωθεί η multisort, σταματάει η χρονομέτρηση, τυπώνεται ο ταξινομημένος πίνακας και στο τέλος εμφανίζονται το μέγεθος N, ο αριθμός νημάτων threads και ο συνολικός χρόνος εκτέλεσης σε δευτερόλεπτα.

Ενδεικτικά τρεξίματα

```
PS C:\Users\vesel\Desktop\openMP> gcc oMP.c -fopenmp -o oMP
PS C:\Users\vesel\Desktop\openMP> ./oMP 100 2
```

A array before sorting:

```
3 77 22 40 21 32 76 58 89 64
24 30 35 27 61 91 60 42 96 36
15 26 17 55 92 82 5 95 13 98
47 11 71 43 4 14 1 39 29 20
85 37 19 38 72 9 70 50 79 68
75 73 8 57 83 59 25 46 16 6
18 33 90 34 49 78 86 63 88 23
74 10 52 7 45 0 87 67 93 2
44 56 97 99 53 66 41 65 69 62
51 94 31 12 54 81 28 84 80 48
```

Sorted array:

```
0 1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17 18 19
20 21 22 23 24 25 26 27 28 29
30 31 32 33 34 35 36 37 38 39
40 41 42 43 44 45 46 47 48 49
50 51 52 53 54 55 56 57 58 59
60 61 62 63 64 65 66 67 68 69
70 71 72 73 74 75 76 77 78 79
80 81 82 83 84 85 86 87 88 89
90 91 92 93 94 95 96 97 98 99
```

Array size N: 100

Number of threads: 2

Execution time: 0.000000000 seconds

```
PS C:\Users\vesel\Desktop\openMP> gcc oMP.c -fopenmp -o oMP
PS C:\Users\vesel\Desktop\openMP> ./oMP 10000 32
```

Array size N: 10000

Number of threads: 32

Execution time: 0.009000063 seconds

Το πρόγραμμα έτρεξε σε Windows 11 με επεξεργαστή AMD Ryzen 7 7700 8-Core Processor, 3801 Mhz, 8 Core(s), 16 Logical Processor(s).

Compile κάναμε με `gcc omp.c -fopenmp -o omp`.